DR BR Ambedkar National Institute of Technology

ARTIFICIAL INTELLIGENCE LAB (IT-515)

Submitted to:

Dr. Simranjeet Singh

Assistant professor

Submitted by:

Granth Gupta

M.Tech-D.A

26201812

Date: 25 August, 2026 LAB-1

---

1. Develop a simple rule-based expert system for animal identification. Define a set of facts and IF–THEN rules and infer the type of animal based on the given facts.

PYTHON CODE

```
# FACTS
facts_test_1 = ["mammal", "carnivore", "strips"]
facts_test_2 = ["bird", "carnivore"]
# RULES
rules = [
    (["mammal", "carnivore", "strips"], "Tiger"),
    (["bird", "carnivore"], "Vulture"),
    (["mammal", "herbivore", "strips"], "Zebra")
]
# Check IF all conditions in a rule match the given facts
def checkTarget(target, rules):
    for conditions, fact in rules:
        if target == fact:
            return conditions
    return []

# ANIMAL IDENTIFICATION FUNCTION
def identifyAnimal(given_facts, rules) -> str:
    for conditions, animal in rules:
        match = True
```

```
        for attribute in conditions:
            if attribute not in given_facts:
                match = False
                break
        if match:
            return animal
    return "Current properties are insufficient to identify the animal"

# TESTING
print("Test 1")
result = identifyAnimal(facts_test_1, rules)
print(result)

print("----------------------------")
print("Test 2")
result = identifyAnimal(facts_test_2, rules)
print(result)
```

OUTPUT

```
Test 1
Tiger
----------------------------
Test 2
Vulture
```

2. Develop an interactive expert system for student course recommendation. The system should ask the user questions about marks, interests, and skills and recommend a suitable course using predefined IF–THEN rules.

PYTHON CODE

```
# FACTS

facts_test_1 = {

    "marks": 80,

    "skill": "logic",

    "interest": "technology"

}

facts_test_2 = {

    "marks": 60,

    "skill": "communication",

    "interest": "business"

}


# RULES

RULES = [

CSE"){"interest": "technology", "skill": "logic", "min_marks": 75}, "BSc in

Biotechnology"){"interest": "healthcare", "skill": "research", "min_marks": 70}, "BSc in

Finance"){"interest": "business", "skill": "logic", "min_marks": 60}, "BBA in

Marketing"){"interest": "business", "skill": "communication", "min_marks": 60}, "BBA in

]


# RECOMMENDATION FUNCTION

def recommendCourse(facts: dict) -> str:
```

```
    for conditions, course in RULES:
        interest_match = facts.get("interest") == conditions["interest"]
        skill_match = facts.get("skill") == conditions["skill"]
        marks_match = facts.get("marks", 0) >= conditions["min_marks"]

        if interest_match and skill_match and marks_match:
            return "Recommended course is " + course

    return "Sorry! I don't have a recommended course for you."

# TESTING
print("Test 1")
result1 = recommendCourse(facts_test_1)
print(result1)

print("---------------------------------")
print("Test 2")
result2 = recommendCourse(facts_test_2)
print(result2)
```

OUTPUT

```
Test 1
Recommended course is BSc in CSE
---------------------------------
Test 2
Recommended course is BBA in Marketing
```

3. Implement a forward-chaining inference engine for an expert system. Given a set of initial facts and IF–THEN rules, repeatedly apply applicable rules to derive new facts until the required conclusion is reached or no further rules can be applied.

PYTHON CODE

```
# KNOWN FACTS

known_facts_1 = ["has_fever", "has_cough"]

known_facts_2 = ["has_cough"]


# RULES

rules = [

    (["has_fever", "has_cough"], "has_flu"),

    (["has_flu"], "needs_rest")

]


def checkFacts(conditions, facts):

    for cond in conditions:

        if cond not in facts:

            return False

    return True


# FORWARD CHAIN

def forward_chaining(facts, rules, goal):

    facts_changed = True

    while facts_changed:
```

```
        facts_changed = False
        for conditions, result in rules:
            if checkFacts(conditions, facts) and result not in facts:
                facts.append(result)
                facts_changed = True
                if result == goal:
                    print(f"Goal '{goal}' successfully reached!")
                    return True
    if goal in facts:
        return True
    else:
        print(f"Could not reach the goal '{goal}'.")
        return False

# TESTING
target1 = "needs_rest"
print("Test 1")

print("Initial Facts: ", known_facts_1)
print("Target: ", target1)
forward_chaining(known_facts_1, rules, target1)
print("-------------------------")



target2 = "has_flu"
print("Test 2")
print("Initial Facts: ", known_facts_2)
print("Targett: ", target2)
```

```
forward_chaining(known_facts_2, rules, target2)
```

OUTPUT

```
Test 1

Initial Facts:  ['has_fever', 'has_cough']

Target:  needs_rest

Goal 'needs_rest' successfully reached!

-------------------------

Test 2

Initial Facts:  ['has_cough']

Target:  has_flu

Could not reach the goal 'has_flu'.
```

4. Develop an expert system for fault diagnosis using backward chaining. Given a suspected fault as the goal, identify the rules and facts required to prove or reject the hypothesis. The system should also display an explanation showing how the conclusion was reached.

PYTHON CODE

```
def checkTarget(target, rules):
    for conditions, fact in rules:
        if target == fact:
            return conditions
    return []
def backward_chaining(facts, rules, goal):
    if goal in facts:
        print(f"Fact '{goal}' is directly known.")
        return True
    conditions = checkTarget(goal, rules)
    if not conditions:
        print(f"No rule or fact found to prove '{goal}'.")
        return False


    print(f"To prove '{goal}', checking required conditions: {conditions}")
    for cond in conditions:
        proven= backward_chaining(facts, rules, cond)
        if not proven:
            print(f"Failed to prove condition '{cond}'. Hypothesis '{goal}'
            rejected.")
            return False
    print(f"All conditions met! Goal '{goal}' is PROVEN.")
```

```
    return True


facts = [
    "Lights are dim",
    "Engine does not turn over"
]
rules = [
    (["Starter clicks", "Lights are dim"], "Dead Battery"),
    (["Engine turns over", "Fuel tank is empty"], "Out of Gas"),
    (["Engine does not turn over"], "Starter clicks")
]
target = "Dead Battery"
is_proven = backward_chaining(facts, rules, target)

print("============================")

print(f"Hypothesis '{target}': {'CONFIRMED' if is_proven else 'REJECTED'}\n")
```

OUTPUT

```
To prove 'Dead Battery', checking required conditions: ['Starter clicks', 'Lights
are dim']
To prove 'Starter clicks', checking required conditions: ['Engine does not
turn over']
Fact 'Engine does not turn over' is directly known.
All conditions met! Goal 'Starter clicks' is PROVEN.
Fact 'Lights are dim' is directly known.
All conditions met! Goal 'Dead Battery' is PROVEN.
============================
Hypothesis 'Dead Battery': CONFIRMED
```